

IDAT



idat

Curso: Desarrollo de Componentes del Negocio

Profesor: Solano Coello, Eloy Iván

Grupo 6:

- Chiara Gutiérrez, Franklin
- Diaz Vilela, Jean Pierre Nicolas
- Morillo Campos, Alexander Rafael
- Ruiz Yamamoto, Fernando Alonso

1. Introducción

En el presente informe técnico se describe la implementación para la gestión de biblioteca universitaria, el desarrollo hecho de una API REST utilizando Spring Boot la cual se encuentra conectada a una base de datos relacional PostgreSQL. El sistema permitirá gestionar libros, usuarios, roles, préstamos, etc. además de incorporar autenticación y autorización utilizando JWT para proteger el acceso a los recursos según el perfil de usuario.

El proyecto está desarrollado desde el punto de vista backend y aplicado una arquitectura por capas en donde no falta persistencia en forma de JPA/Hibernate, validación de datos y un manejo centralizado de errores, así como la documentación y prueba de los endpoints usando Postman.

2. Objetivos del Proyecto

La aplicación debe ser capaz de:

- Registrar y gestionar libros, usuarios y préstamos de biblioteca
- Conectarse con una base de datos relacional mediante JPA, Hibernate y Spring Data
- Implementar la seguridad con JWT para la autenticación sin estado
- Proteger las rutas según el rol de usuario (ADMIN y USER)
- Validar los datos de entrada y centralizar el manejo de errores

3. Descripción General del Sistema del Sistema

El proyecto es una API REST encargada de la gestión de bibliotecas, cuyo objetivo es facilitar el control de los recursos bibliográficos y su seguimiento para realizar préstamos a los usuarios que se encuentran registrados.

El sistema permite:

- Registrar libros con datos como: título, autor, stock, fecha de publicación, etc.
- Registrar usuarios con credenciales de acceso y asignar roles sobre los mismos.
- Administrar roles para controlar permisos de acceso.
- Registrar préstamos a usuarios de libros.
- Controlar la disponibilidad de stock al momento de realizar un préstamo.
- Administrar la autenticación mediante login y emisión de token JWT.
- Restringir operaciones amparadas en los permisos del rol autenticado.

4. Tecnologías Implementadas

- Java 21
- Spring Boot 3
- Spring Web
- Spring Data JPA
- Hibernate
- Spring Security
- JWT
- PostgreSQL
- Lombok
- MapStruct
- Swagger / OpenAPI
- Postman
- Maven

5. Arquitectura del Proyecto

Esta organización fue llevada a cabo utilizando una arquitectura por capas, que permite tener una clara separación de responsabilidades.

- Controller: expone los endpoint REST.
- Service: implementa la lógica de negocio.
- Repository: realiza el acceso a la base de datos.
- Entity: representa las tablas de la base de datos.
- DTO: definir objetos de entrada y salida.
- Mapper: transformar entidad a DTO y viceversa.
- Security: implementar autenticación, JWT y control de acceso.
- Exception: centralizar el manejo de errores.

Esta estructura aumenta la mantenibilidad, escalabilidad y legibilidad del sistema.

6. Módulos desarrollados

6.1. Authentication

El sistema define los endpoints de registro, así como de login.

Al iniciar la sesión un usuario de forma correcta se genera un token JWT que debe ser enviado en las peticiones que merecen estar protegidas.

Funciones:

- Registro de usuario.
- Login.
- Generar y verificar un token JWT.
- Cargar el usuario autenticado con roles.

6.2. Gestión de los usuarios

Permite:

- Crear usuarios.
- Listar usuarios.
- Consultar un usuario según su Id.
- Modificar usuarios.
- Desactivar usuarios lógicamente.

6.3. Gestión de roles

Permite:

- Crear roles.
- Listar roles.
- Consultar un rol por el Id.
- Modificar roles.
- Desactivar roles lógicamente.

Además, si no existen en la base de datos los roles base ADMIN y USER se crean al inicio de la aplicación.

6.4. Gestión de Libros

Permite:

- Registrar libros.
- Listar libros.
- Buscar libros por Id.
- Actualizar información sobre libros.
- Desactivar libros.
- Controlar el stock disponible.

6.5. Gestión de Préstamos

Permite:

- Registrar préstamos.
- Listar préstamos.
- Buscar préstamos por Id.
- Actualizar préstamos.
- Desactivar préstamos.

Regla básica de negocio:

Un préstamo se podrá registrar solamente si hay stock disponible. Al registrar un préstamo, se debe decrementar el stock del libro por unidad.

7. Seguridad del Sistema

La seguridad fue implementada en base a Spring Security y JWT.

Las principales características incluyen:

- Autenticación
- Generación del token JWT después del login
- Validación del token en cada petición protegida
- Restricción del acceso de acuerdo a los roles correspondientes

Reglas de acceso implementadas:

- ADMIN pueden gestionar libros, préstamos, usuarios y roles.
- USER puede consultar libros y registrar o consultar préstamos siguiendo las reglas definidas. Las rutas públicas corresponden al módulo de autenticación y de documentación (Swagger).

8. Base de Datos

La aplicación utiliza como sistema de gestión de base de datos relacional a PostgreSQL.

La conexión se realiza desde Spring Boot mediante la configuración de la aplicación en archivos application.yaml donde se utiliza JPA/Hibernate para el mapeo objeto-relacional.

Las principales entidades son:

- Usuario
- Rol
- Libro
- Prestamo

Las principales relaciones son:

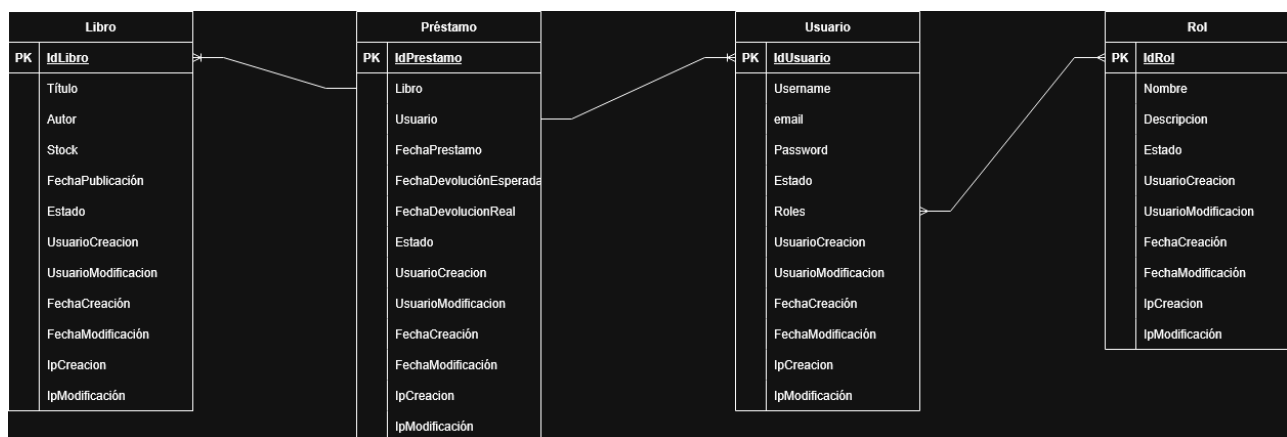
- Un usuario puede tener varios roles.
- Un rol puede pertenecer a varios usuarios.
- Un usuario puede tener varios préstamos.
- Un libro puede tener muchos préstamos.
- Cada préstamo pertenece a un usuario y a un libro.

9. Diagrama de Entidades

Se tienen 4 entidades: Libro, Préstamo, Usuario y Rol.

Libro y Préstamo tienen una relación de 1:N, de igual manera Usuario y Préstamo.

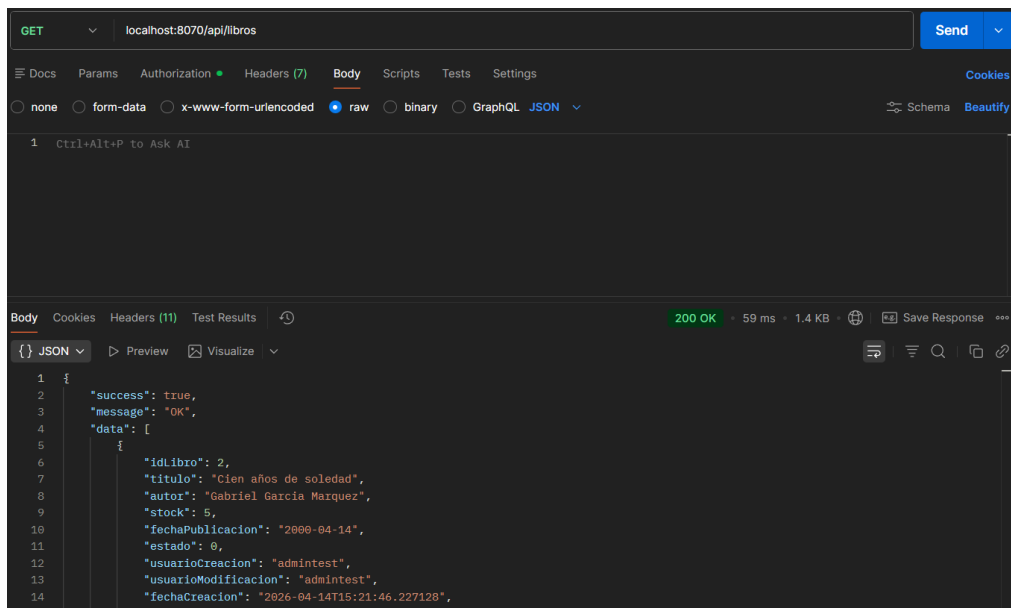
Usuario y Rol tiene una relación N:N por lo que se crea una tabla de detalle intermedia.



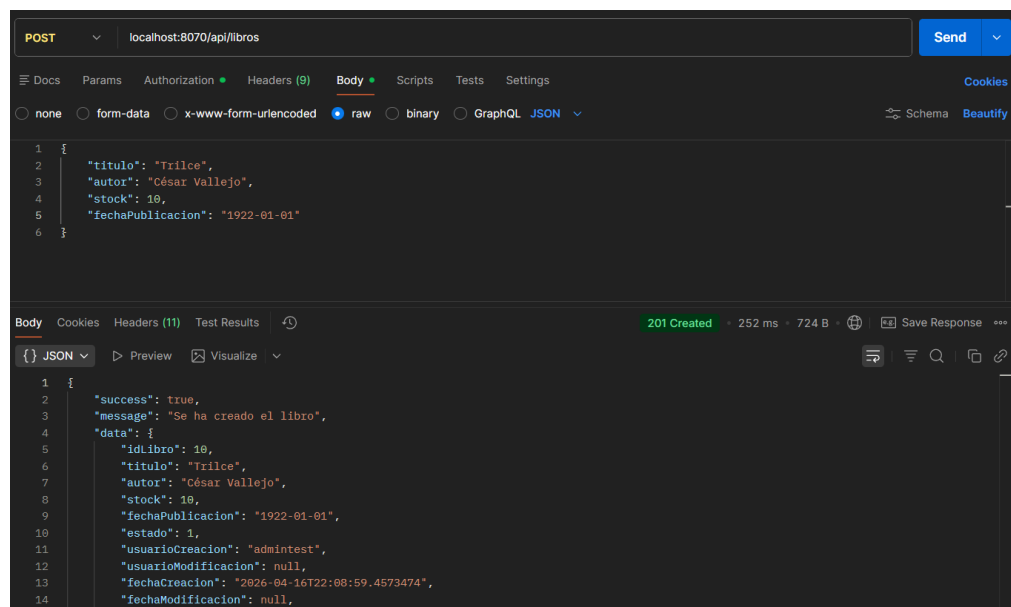
10. Evidencias de pruebas con Postman

Las pruebas funcionales del sistema se llevaron a cabo mediante Postman comprobando así la correcta respuesta a los endpoints, la autenticación mediante JWT, la autorización por roles, la validación de datos y el tratamiento de errores. Las evidencias muestran respuestas correctas y los escenarios de rechazo controlado que confirman el correcto funcionamiento de la API.”

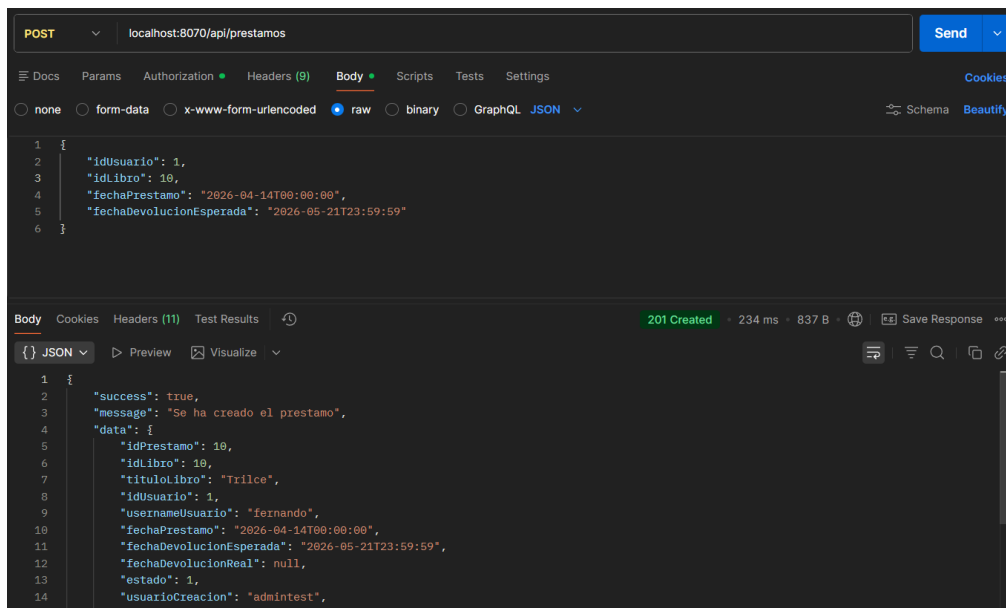
Prueba 1: Registro correcto de usuario.



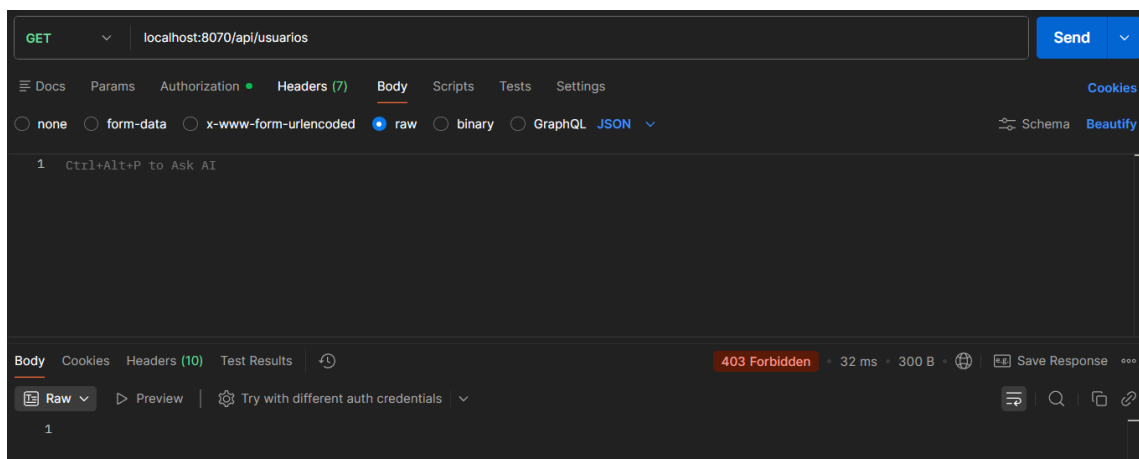
Prueba 4: Registro de libro con rol ADMIN.



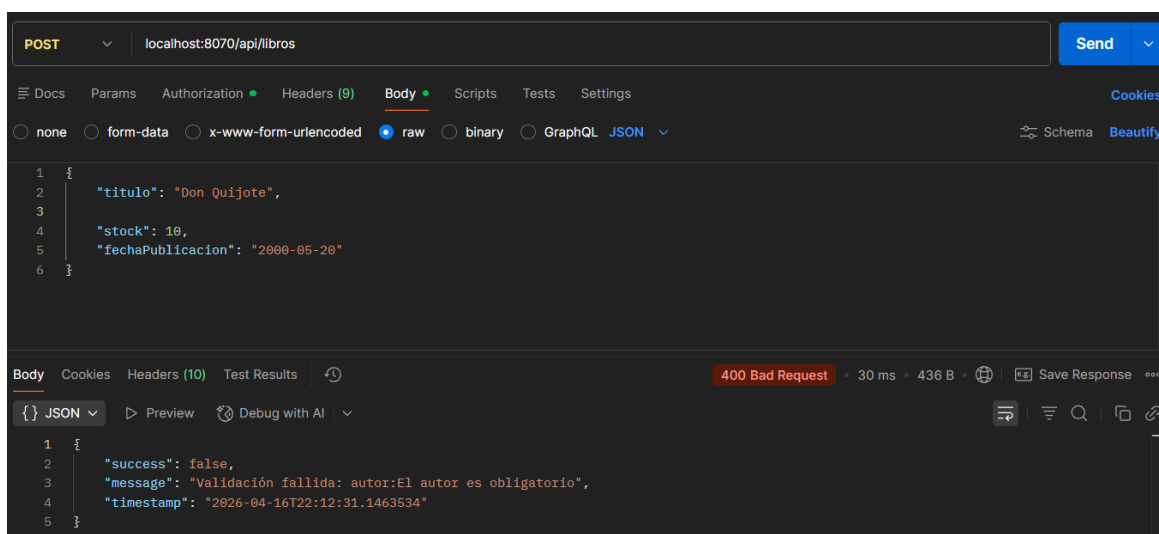
Prueba 5: Registro de préstamo con stock disponible.



Prueba 6: Validación de acceso denegado para usuario sin permisos.



Prueba 7: Validación de errores por datos incompletos o no válidos.



11. Repositorio del proyecto

El código fuente del proyecto se encuentra publicado en GitHub para su revisión por parte del profesor.

Enlace del repositorio: <https://github.com/fernandoruiz2016/API-Biblioteca-SpringBoot>

12. Instrucciones para ejecutar

Para poder ejecutar el sistema, se debe contar con Java 21, PostgreSQL y Maven o un IDE que sea compatible con Spring Boot

Una vez que se clona la aplicación, se configura la conexión a la base de datos en el archivo application.yaml y se arranca la aplicación. Una vez levantado el servicio, se pueden probar los endpoints desde Postman.

Para ejecutar el proyecto se debe:

- Clonar el repositorio desde GitHub.
- Configurar la base de datos PostgreSQL.
- Crear la base de datos y schema.
- Verificar usuario, contraseña y puerto en application.yaml
- Ejecutar la aplicación con Maven o bien desde el IDE correspondiente.
- Ejecutar endpoints desde Postman.

13. Conclusiones

Se logró desarrollar una API REST que funciona correctamente para cubrir los requerimientos de una biblioteca universitaria; se ha desarrollado de manera eficaz la persistencia en base de datos relacional; se incluye la autenticación a través del estándar JWT, la gestión de los permisos por roles y de la validación de los datos. La solución implementada responde a los requerimientos principales del trabajo, permitiendo tener una buena base de desarrollo para posibles ampliaciones futuras, tales como devoluciones con mayor detalle, reportes, histórico de préstamos y mejoras en la realización de pruebas.